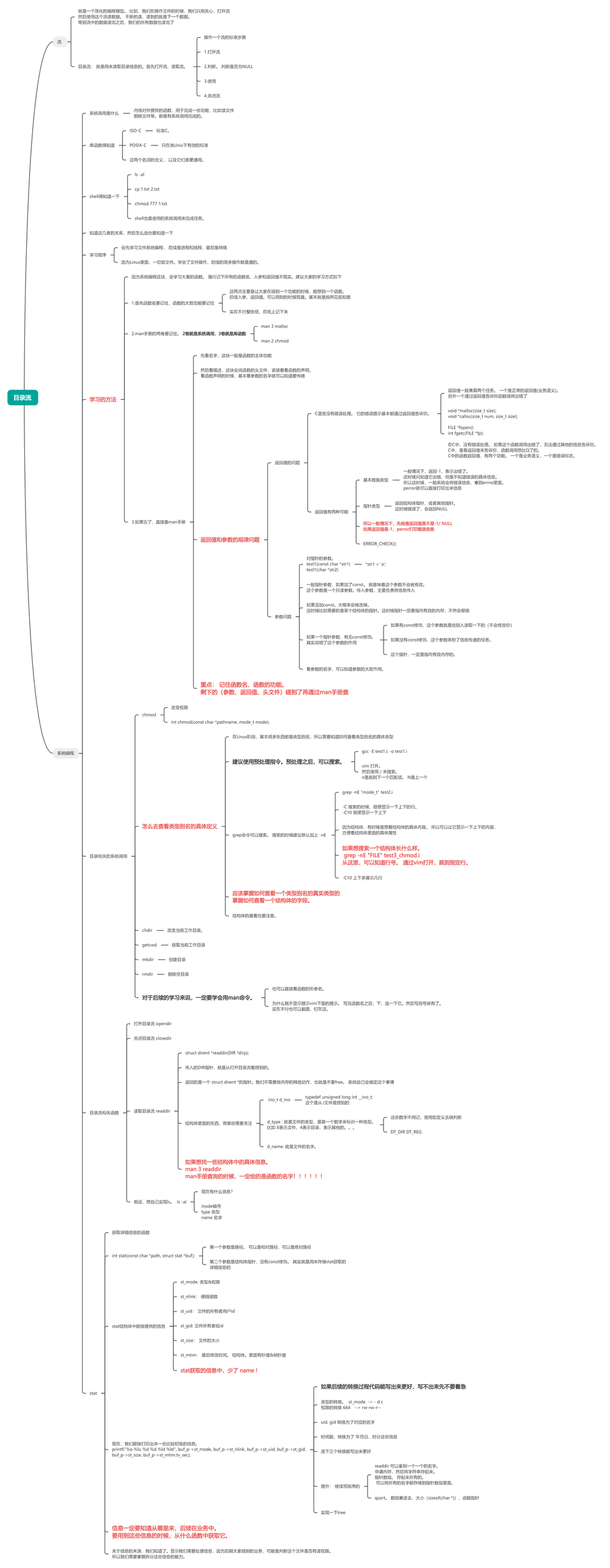
目录流
流
就是一个简化的编程模型。 比如，我们在操作文件的时候，我们只用关心，打开流
然后使用这个流读数据。 不断的读，读到的就是下一个数据。
等到完中的数据读完之后，我们的所有数据也读完了
目录流： 就是用来读取目录信息的。首先打开流，读取流。
操作一个流的标准步骤
1.打开流
2.判断， 判断是否为NULL
3.使用
4.关闭流
系统编程
系统调用是什么
内核对外提供的函数，用于完成一些功能，比如读文件
删除文件等。都是有系统调用完成的。
库函数得知道
ISO-C 标准C。
POSIX-C 只在类Unix下有效的标准
这两个名词的含义， 以及它们谁更通用。
shell得知道一下
ls -al
cp 1.txt 2.txt
chmod 777 1.txt
shell也是使用的系统调用来完成任务。
知道这几者的关系，然后怎么选也要知道一下
学习顺序
会先学习文件系统编程， 后续是进程和线程，最后是网络
因为Linux里面，一切皆文件。学会了文件操作，后续的很多操作都是通的。
学习的方法
因为系统编程这块，会学习大量的函数。 强行记下所有的函数名、入参和返回值不现实。建议大家的学习方式如下
1.首先函数名要记住，函数的大致功能要记住
这两点主要是让大家在碰到一个功能的时候，能想到一个函数。
后续入参、返回值。可以用到的时候现查。基本就是按照见名知意
实在不行整张纸，在纸上记下来
2.man手册的两卷要记住。 2卷就是系统调用，3卷就是库函数
man 3 malloc
man 2 chmod
3.如果忘了，直接查man手册
先看名字，这块一般是函数的主体功能
然后看描述，这块会说函数的头文件，紧接着看函数的声明。
看函数声明的时候，基本看参数的名字就可以知道要传啥
返回值和参数的规律问题
返回值的问题
C语言没有错误处理。 它的错误提示基本都通过返回值告诉你。
返回值一般兼具两个任务， 一个是正常的返回值(业务语义)。
另外一个通过返回值告诉你函数调用出错了
void *malloc(size_t size);
void *calloc(size_t num, size_t size);
FILE *fopen()
int fgetc(FILE *fp);
在C中，没有错误处理。 如果这个函数调用出错了，无法通过其他的信息告诉你。
C中，是靠返回值来告诉你，函数调用完以了的。
C中的函数返回值，有两个功能。 一个是业务语义，一个是错误标志。
返回值有两种可能
基本数据类型
一般情况下，返回-1，表示出错了。
这时候只知道它出错，但是不知道错误的具体信息。
所以这时候，一般系统会将错误信息，塞到errno里面。
perror就可以直接打印出来信息
指针类型
返回结构体指针，或者其他指针。
这时候错误了，会返回NULL
所以一般情况下，先检查返回值是不是-1/ NULL
如果返回值是-1，perror打印错误信息
ERROR_CHECK();
参数问题
对指针的参数。
test1(const char *str1) *str1 = 'a';
test1(char *str2)
一般指针参数，如果加了const， 就意味着这个参数不会被修改。
这个参数是一个只读参数。传入参数，主要负责将信息传入
如果没加const，大概率会被改掉。
这时候比如需要的是某个结构体的指针，这时候指针一定要指向有效的内存，不然会报错
如果一个指针参数，有无const修饰。
其实说明了这个参数的作用
如果有const修饰，这个参数就是给别人读取一下的（不会修改你）
如果没有const修饰，这个参数承担了信息传递的任务。
这个指针，一定是指向有效内存的。
看参数的名字，可以知道参数的大致作用。
重点： 记住函数名、函数的功能。
剩下的（参数、返回值、头文件）碰到了再通过man手册查
目录相关的系统调用
chmod
改变权限
int chmod(const char *pathname, mode_t mode);
怎么去查看类型别名的具体定义
在Linux阶段，基本很多东西都是类型别名，所以需要知道如何查看类型别名的具体类型
建议使用预处理指令。预处理之后，可以搜索。
gcc -E test1.c -o test1.i
vim 打开。
然后使用 / 来搜索。
n是跳到下一个匹配项。 N是上一个
grep命令可以搜索。 搜索的时候建议默认加上 -nE
grep -nE "mode_t" test2.i
-C 搜索的时候，顺便显示一下上下的行。
-C10 顺便显示一下上下
因为结构体，有时候是想看结构体的具体内容。 所以可以让它显示一下上下的内容，
方便看结构体里面的具体属性
如果想搜索一个结构体长什么样。
grep -nE "FILE" test3_chmod.i
从这里，可以知道行号。 通过vim打开，跳到指定行。
-C10 上下多展示几行
应该掌握如何查看一个类型别名的真实类型的
掌握如何查看一个结构体的字段。
结构体的查看也要注意。
chdir 改变当前工作目录。
getcwd 获取当前工作目录
mkdir 创建目录
rmdir 删除空目录
对于后续的学习来说，一定要学会用man命令。
也可以直接看函数的形参名。
为什么我不显示提示vim下面的提示。 写完函数名之后，下，选一下它。然后写括号就有了。
实在不行也可以截图，钉在这。
目录流相关函数
打开目录流 opendir
关闭目录流 closedir
读取目录流 readdir
struct dirent *readdir(DIR *dirp);
传入的DIR指针，就是从打开目录流看捞到的。
返回的是一个 struct dirent *的指针。我们不需要做内存的释放动作，也就是不要free。 系统自己会搞定这个事情
结构体里面的东西，有哪些需要关注
ino_t d_ino
typedef unsigned long int __ino_t;
这个是从 i文件里捞到的
d_type：就是文件的类型，是靠一个数字来标识一种类型。
比如 8表示文件，4表示目录，表示其他的。。。
这些数字不用记，使用宏定义去做判断
DT_DIR DT_REG
d_name: 就是文件的名字。
如果想找一些结构体中的具体信息。
man 3 readdir
man手册查询的时候，一定给的是函数的名字！！！！！
假设，想自己实现ls -al
现在有什么信息？
inode编号
type 类型
name 名字
stat
获取详细信息的函数
int stat(const char *path, struct stat *buf);
第一个参数是路径。 可以是相对路径，可以是绝对路径
第二个参数是结构体指针，没有const修饰。 其实就是用来存储stat获取的
详细信息的
stat结构体中能够提供的信息
st_mode: 类型&权限
st_nlink: 硬链接数
st_uid: 文件的所有者用户id
st_gid: 文件所有者组id
st_size: 文件的大小
st_mtim: 最后修改时间。 结构体。里面有秒值&纳秒值
stat获取的信息中，少了 name！
现在，我们能够打印出来一些比较初级的信息。
printf("%o %lu %d %d %ld %ld", buf_p->st_mode, buf_p->st_nlink, buf_p->st_uid, buf_p->st_gid, buf_p->st_size, buf_p->st_mtim.tv_sec);
如果后续的转换过程代码能写出来更好，写不出来先不要着急
类型的转换， st_mode -> - d c
权限的转换 664 --> rw-rw-r--
uid, gid 转换为了对应的名字
时间戳，转换为了 年月日，时分这些信息
底下三个转换能写出来更好
提升： 继续写排序的
readdir 可以拿到一个一个的名字。
申请内存，然后将字符串存起来。
指针数组。 存起来所有的。
可以将所有的名字都存储到指针数组里面。
qsort。 数组塞进去，大小（sizeof(char *)），函数指针
实现一下tree
信息一定要知道从哪里来，后续在业务中。
要用到这些信息的时候，从什么函数中获取它。
关于信息的来源，我们知道了。显示我们需要处理信息。因为后期大家碰到的业务，可能是判断这个文件是否有读权限。
所以我们需要掌握拆分这些信息的能力。